

Arhitecturi Semantice și Baze de date agnostice

Conf. Dr.Cristian KEVORCHIAN
Universitatea din București

Arhitecturi Semantice

- O **Arhitectură Semantică** reprezintă structura unui sistem de gestionare a datelor și informațiilor care **pune accentul pe înțelesul (semantica) datelor**, nu doar pe structura lor fizică sau logică.
- **Scopul:** este acela de a permite sistemelor (și agenților inteligenți) să înțeleagă, să interpreteze și să utilizeze datele de o manieră inteligentă cu potențial de automatizare automatizate.
- **Elemente cheie:**
 - **Ontologii:** Reprezintă o schemă formală de concepte și relații într-un domeniu specific (de exemplu, o ontologie medicală sau financiară). Acestea oferă un **cadru comun** de înțelegere a datelor.
 - **Metadata Semantic:** Datele sunt îmbogățite cu informații care descriu ce reprezintă ele, nu doar cum sunt structurate.
 - **LLM ca Language-Inspired Computing (Bazat pe Modele Statistice)** - Sunt sisteme capabile să înțeleagă, să genereze și să "raționeze" pe baza limbajului natural, învățând din cantități uriașe de text și date.
- LLM-urile pot acționa ca un strat semantic emergent între date brute și ontologii. În practică, cele două abordări se completează: LLM-urile accelerează și automatizează procesul de construire semantică, în timp ce ontologiile oferă rigoare și structură.

Fluxul de Lucru Semantic-Numeric Modern

Date Numerice Brute → Filozofie
Computațională (LangExtract) →
Ontologie (Context Formal) → LLM
(Rezolvarea Problemei)



Datele Sintetizate Numeric (Realitatea Brute)

- **Intrare:** Seturi de date numerice mari, serii de timp, înregistrări de tranzacții etc., care, luate izolat, au un sens limitat.
- **Realitatea:** Reprezintă o stare de fapt (ex. "Vânzările au crescut cu 10%"). Această realitate este **implicit** numerică.

Filozofia Computațională (LangExtract)

- Acesta este motorul care extrage sensul. Dacă presupunem că **LangExtract** este o modalitate avansată de prelucrare, inclusiv cu tehnici inspirate de LLM, el îndeplinește două funcții esențiale:
- **Descifrarea:** Folosește capacitățile de înțelegere lingvistică/semantică (chiar dacă pornește de la date numerice) pentru a recunoaște **tiparele și entitățile relevante** din date. De exemplu, convertește "seria temporală X a crescut" în conceptul semantic "Trend Pozitiv de Creștere".
- **Generarea Ontologiei (Contextului):** Pe baza acestei descifrări, LangExtract **generează sau adaptează** o ontologie care formalizează contextul realității numerice.

Ontologia (Stratul Intermediar de Context Formal)

- Ontologia creată în această etapă nu este un scop în sine, ci o **formă de reprezentare structurată a problemei**.
- **Scopul:** De a transforma sensul **implicit** extras de LangExtract într-un format **explicit, formal și consistent** (similar cu OWL/RDF sau un Graf de Cunoștințe) pe care LLM-ul îl poate procesa eficient.
- **Exemplu:** Ontologia ar putea defini: (Obiect) Produsul X *are o relație* (Proprietate) Trend de Creștere *cu valoarea* (Valoare) 10%.

LLM-ul (Rezolvarea Problemei)

- Ontologia (sub forma unui context structurat, logic și precis) este trimisă către LLM.
- **Input pentru LLM:** Nu datele brute, ci **descrierea formalizată și semantică a realității**.
- **Sarcina LLM:** De a efectua **raționamentul final** sau de a **genera răspunsul** (acțiune, decizie, explicație) pe baza contextului precis primit.

Agnosticismul Modelelor de date in Arhitecturile Semantice

- Pentru ca o **Arhitectură Semantică** să funcționeze eficient într-un mediu de **Cloud Computing**, care este inerent eterogen (multi-cloud, hibrid, multiple servicii de stocare), este esențial ca datele să fie **agnostic** față de infrastructura subiacentă.
- Scopul este de a separa **înțelegerea (semantica)** și **logica de business** de **detaaliile tehnice** de stocare și localizare, care se realizează prin **agnosticismul datelor** la nivel de cloud se prin implementarea unui **strat de abstractizare** între sursele fizice (datele din AWS S3, Azure Data Lake, Google Cloud Storage, baze de date specifice furnizorilor) și consumatorii semantici (ontologia/LLM):
 - 1. Stratul de Virtualizare/Abstractizare a Datelor:** Acesta acționează ca un *proxy* sau un *interfață uniformă*. El ascunde complexitatea interogărilor specifice fiecărui furnizor (ex. limbaje diferite, API-uri diferite) și expune datele într-un format și o interfață standard (ex. SQL sau API-uri REST).
 - 2. Modelarea Semantică (Ontologia):** Ontologia sau Graficul de Cunoștințe (eventual generat sau îmbunătățit de LLM) interacționează **doar** cu acest strat virtualizat. Ea definește concepte de business (ex. "Client") fără a ști dacă acel client este stocat în baza de date Oracle a unui Cloud sau într-un tabel NoSQL al altui Cloud.
 - 3. Containerizare și Standarde Deschise:** Folosirea tehnologiilor precum **Kubernetes** pentru a rula aplicațiile de date și procesele (parte din DOaaS-Data Operation as a Service) și a formatelor de date deschise (ex. Parquet, Apache Avro) asigură portabilitatea logică și fizică între furnizori.



Modele de date – Definiție

Modelele de Date sunt structuri conceptuale care definesc modul în care datele sunt organizate, stocate și gestionate într-o bază de date. Ele oferă un cadru pentru a înțelege relațiile dintre diferitele tipuri de date și pentru a facilita accesul și manipularea acestora.

Modele de date – Exemple(I)

1. Modelul Relational

Descriere: Organizează datele în tabele (relații) formate din rânduri și coloane. Fiecare tabel reprezintă o entitate, iar fiecare rând reprezintă o instanță a acelei entități.

Exemple: MySQL, PostgreSQL, Microsoft SQL Server.

Utilizare: Potrivit pentru aplicații care necesită consistență și integritate a datelor, cum ar fi sistemele financiare și de ERP sau CRM.

2. Modelul Document

Descriere: Stochează datele sub formă de documente JSON, BSON sau XML, cu structuri complexe și încorporate.

Exemple: MongoDB, CouchDB, Azure Cosmos DB (Document API).

Utilizare: Ideal pentru aplicații care necesită flexibilitate în schema datelor, cum ar fi aplicațiile web și mobile.

3. Modelul "Key-Value"

Descriere: Stochează datele sub formă de perechi cheie-valoare, unde fiecare cheie este un identificator unic pentru valoarea asociată.

Exemple: Redis, DynamoDB, Riak.

Utilizare: Potrivit pentru cazuri de utilizare care necesită acces rapid la date printr-o cheie unică, cum ar fi cache-uri și sesiuni utilizator.

Modele de date – Example(II)

4. Modelul Column-Family

Descriere: Stochează datele în coloane grupate în familii de coloane, permițând accesul eficient la datele aferente.

Exemple: Apache Cassandra, HBase, ScyllaDB.

Utilizare: Ideal pentru aplicații care necesită scrieri și citiri rapide la scară mare, cum ar fi sistemele de recomandare și analiza datelor.

5. Modelul Graf

Descriere: Stochează datele sub formă de noduri și muchii, reprezentând relațiile dintre entități.

Exemple: Neo4j, ArangoDB, Azure Cosmos DB (Gremlin API).

Utilizare: Ideal pentru aplicații care necesită gestionarea relațiilor complexe, cum ar fi rețelele sociale și sistemele de gestionare a fraudelor.

6. Modelul Obiect

Descriere: Stochează datele sub formă de obiecte, similar cu programarea orientată pe obiecte, unde fiecare obiect conține date și metode.

Exemple: db4o, ObjectDB.

Utilizare: Potrivit pentru aplicații care utilizează programarea orientată pe obiecte și necesită o mapare directă între obiectele din cod și datele stocate.

Modele si Structuri de Date

Model de Date	Structuri de Date Utilizate
Relational	Tabele, Matrice
Document	Liste, Dicționare, Tensori
Key-Value	Hash Tables (Tabele de dispersie), Dicționare
Column-Family	Matrice, Tabele
Graf	Grafuri, Liste de adiacență, Matrice de adiacență
Obiect	Obiecte, Liste, Dicționare
Secvențe și Serii Temporale	Liste, Tabele, Serii Temporale

Utilizarea structurilor de date

Relational: Utilizează tabele pentru a organiza datele în rânduri și coloane, iar matricile sunt folosite pentru operațiuni matematice și analize.

Document: Listele și dicționarele sunt structuri de date comune pentru stocarea documentelor JSON sau BSON, iar tensorii sunt utilizați pentru procesarea datelor în rețele neuronale.

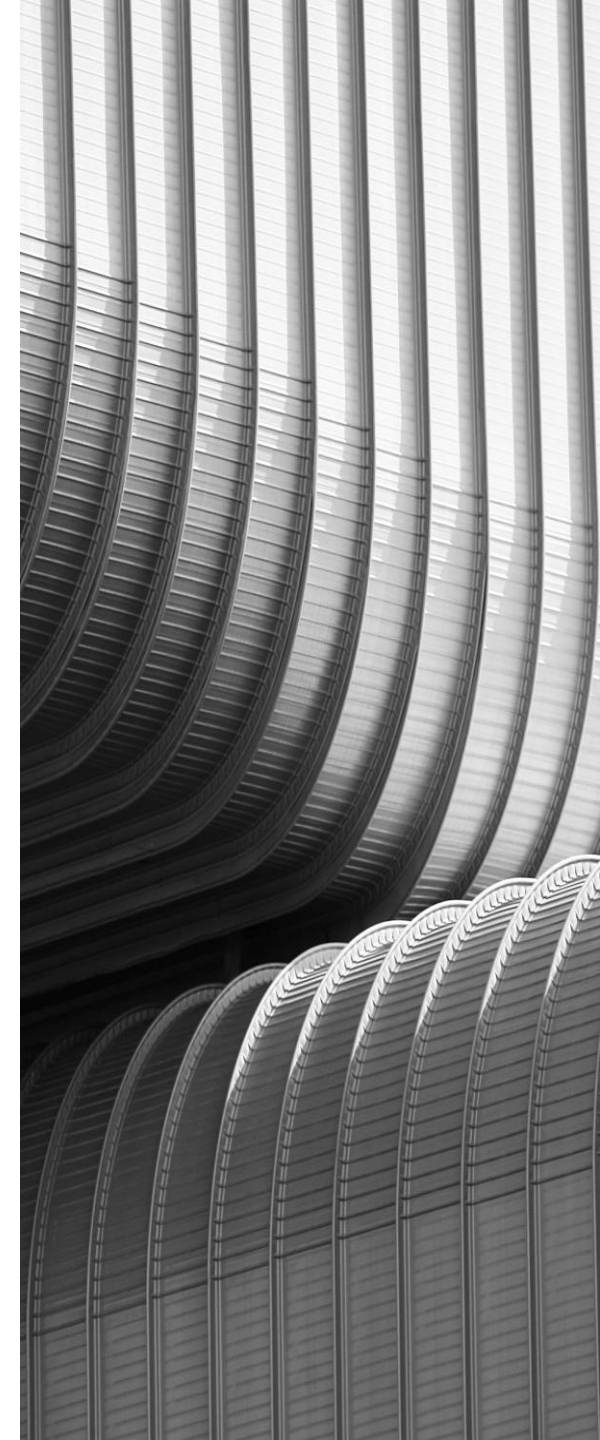
Key-Value: Hash tables și dicționarele permit acces rapid la date prin chei unice.

Column-Family: Matricele și tabelele sunt utilizate pentru a stoca datele în coloane grupate, optimizând accesul la datele asociate.

Graf: Grafurile, listele de adiacență și matricile de adiacență sunt structuri de date esențiale pentru reprezentarea relațiilor complexe dintre entități.

Obiect: Obiectele, listele și dicționarele sunt utilizate pentru a stoca datele și comportamentele asociate în programarea orientată pe obiecte.

Secvențe și Serii Temporale: Listele, tabelele și seriile temporale sunt utilizate pentru a gestiona și analiza datele ordonate în timp.



Baze de date agnostice

Bazele de date agnostice sunt sisteme de gestiune a bazelor de date care nu sunt legate de un anumit model de date sau limbaj de interogare specific. Acestea sunt proiectate pentru a fi flexibile și pentru a suporta multiple modele de date și limbaje de interogare, permițând utilizatorilor să aleagă cel mai potrivit instrument pentru nevoile lor.

Categorii de baze de date agnostice

- 1. Agnostice de schema:** Aceste baze de date nu impun o structură fixă pentru date, permițând flexibilitate în modul în care datele sunt stocate și gestionate. Exemple includ bazele de date NoSQL, cum ar fi MongoDB și Cassandra, care pot gestiona date semi-structurate sau nestructurate.
- 2. Agnostice de limbaj de programare:** Aceste baze de date pot fi accesate și manipulate folosind diverse limbaje de programare, fără a fi legate de un anumit limbaj. De exemplu, PostgreSQL și MySQL pot fi utilizate cu limbaje precum Python, Java, C#, și multe altele.
- 3. Agnostice de model de date:** Aceste baze de date suportă mai multe modele de date într-un singur sistem, permițând utilizatorilor să aleagă cel mai potrivit model pentru nevoile lor. Un exemplu este ArangoDB, care suportă modele de date relaționale, documente și grafuri.

Baze de date agnostice expuse ca serviciu

Bază de date	Caracteristici principale	Platformă de cloud
Azure Cosmos DB	Multi-model (documente, grafuri, coloane, cheie-valoare), distribuție globală, scalabilitate automată	Microsoft Azure
Amazon DynamoDB	Serverless, NoSQL (cheie-valoare, documente), performanță ridicată	Amazon Web Services (AWS)
Google Cloud Firestore	NoSQL document-oriented, sincronizare în timp real, suport offline	Google Cloud
IBM Cloudant	NoSQL (schema-free JSON), scalabilitate automată, disponibilitate ridicată	IBM Cloud
Oracle Autonomous Database	Automatizare completă, suport multi-model, scalabilitate elastică	Oracle Cloud
MongoDB Atlas	Multi-cloud (AWS, Azure, Google Cloud), model de date document, automatizare și optimizare a resurselor	AWS, Azure, Google Cloud
Amazon Aurora	Compatibilitate cu MySQL și PostgreSQL, performanță ridicată, scalabilitate automată	Amazon Web Services (AWS)

Baze de date agnostice multi-model

Bazele de date agnostice multi-model sunt sisteme de gestionare a bazelor de date care suportă mai multe modele de date într-un singur sistem integrat. Acestea permit stocarea și gestionarea diferitelor tipuri de date, cum ar fi relaționale, documente, grafuri și cheie-valoare, într-o singură platformă.

Baze de date multi-model. Caracteristici

1. Suport Multi-Model:

Pot gestiona diferite tipuri de date, cum ar fi documente, grafuri, key-value, și relaționale, într-o singură platformă.

Exemple: Azure Cosmos DB, ArangoDB, OrientDB.

2. Flexibilitate în Limbajul de Interogare:

Permite utilizarea mai multor limbaje de interogare, cum ar fi SQL, Gremlin, MongoDB Query Language, etc.

Utilizatorii pot alege limbajul de interogare care se potrivește cel mai bine cu cerințele lor.

3. Scalabilitate și Performanță:

Proiectate pentru a scala automat și pentru a oferi performanță ridicată indiferent de modelul de date utilizat.

Pot gestiona volume mari de date și cerințe de performanță variabile.

4. Distribuție Globală:

Oferă replicare și distribuție globală automată, asigurând disponibilitate și latență scăzută la nivel global.

Exemple: Azure Cosmos DB.

Azure CosmosDB

Azure Cosmos DB este un serviciu de bază de date multi-model, gestionat complet de platforma de cloud computing Microsoft Azure. Este proiectat pentru a oferi disponibilitate ridicată, scalabilitate și acces cu latență redusă la date pentru *aplicațiile moderne*.

Ideea de *aplicație modernă*, în acest context, face referință la o familie de aplicații care sunt proiectate pentru a fi scalabile, distribuite global și capabile să gestioneze volume mari de date cu performanță ridicată și latență redusă.

Acest tip de aplicații sunt construite pentru a funcționa în cloud și beneficiază de caracteristicile avansate oferite de Azure Cosmos DB.

Azure CosmosDB - Modele de date

1. Modelul de documente:

1.Descriere: Stocază datele sub formă de documente JSON. Este ideal pentru aplicații care necesită flexibilitate în structura datelor.

Exemplu: MongoDB API, SQL API

2. Modelul de grafuri:

1.Descriere: Permite stocarea și interogarea datelor sub formă de noduri și muchii, fiind ideal pentru aplicații care necesită gestionarea relațiilor complexe între date.

Exemplu: Gremlin API

3. Modelul de coloane:

1.Descriere: Stocază datele în tabele cu rânduri și coloane, similar cu bazele de date relaționale, dar optimizat pentru interogări de tip OLAP (Online Analytical Processing).

Exemplu: Cassandra API

4. Modelul cheie-valoare:

1.Descriere: Stocază datele sub formă de perechi cheie-valoare, fiind ideal pentru aplicații care necesită acces rapid la date printr-o cheie unică.

Exemplu: Table API

API-uri suportate

SQL API: Permite interogarea datelor folosind un dialect SQL prietenos cu JSON

MongoDB API: Compatibil cu protocolul MongoDB, permițând utilizarea driverelor MongoDB

Gremlin API: Suportă interogări de grafuri folosind limbajul Gremlin

Cassandra API: Compatibil cu limbajul de interogare Cassandra (CQL)

Table API: Suportă modelul de date cheie-valoare2

Caracteristică	Azure Cosmos DB	Amazon DynamoDB
Modele de date	Documente, grafuri, coloane, cheie-valoare	Cheie-valoare, documente
Distribuție globală	Da, replicare automată în mai multe regiuni Azure	Da, replicare globală cu Global Tables
Scalabilitate	Scalabilitate automată, ajustare dinamică a capacității	Scalabilitate automată, moduri de capacitate provisionată și la cerere
Performanță	Latențe sub 10 ms pentru citiri și sub 15 ms pentru scrieri	Latențe de ordinul milisecundelor. DAX pentru îmbunătățirea performanței
Consistență	Cinci niveluri de consistență: eventuală, prefix consistent, sesiune, stalență limitată, strictă	Consistență eventuală și strictă
Securitate	Criptare SSL/TLS, control bazat pe roluri (RBAC), integrare cu Azure AD(Entra)	Criptare, autentificare IAM, criptare KMS
Backup-uri	Backup-uri continue și periodice	Backup-uri la cerere și recuperare punctuală (PITR)
Prețuri	Model de tarificare pe baza unităților de cerere (RU) și model serverless	Model de tarificare pe baza unităților de cerere (RCU/WCU) și costuri suplimentare pentru funcții adiționale
API-uri suportate	SQL, MongoDB, Cassandra, Gremlin, Table	Documente și cheie-valoare



DEMO